Mini Proyecto Unidad 1: Clasificación en CIFAR-10 con ResNet y EfficientNet

- Pipeline completo de clasificación con CNN (ResNet o EfficientNet) sobre CIFAR-10 (Partes 1 a 5)
- Comparativa técnica de dos arquitecturas CNN con curvas de aprendizaje y métricas (Partes 5 y 6) *Reporte de diseño (arquitectura, hiperparámetros, estrategia de entrenamiento) (Parte 7)

Recomendación de entorno

Activen GPU en Colab: **Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU**. ResNet-18 y EfficientNet-B0 a resolución 224×224 son considerablemente más pesados que la CNN pequeña de la práctica anterior; en CPU van a funcionar, pero mucho más lento. Las constantes `TAM_TRAIN`, `TAM_VAL` y `N_EPOCAS` están pensadas para ajustarse según el tiempo y el hardware disponible.

0. Configuración inicial

In [1]:
```
%pip install -q torchmetrics 2>/dev/null || true
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import time

%matplotlib inline
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import pandas as pd

from torchmetrics.classification import (
    MulticlassAccuracy,
    MulticlassPrecision,
    MulticlassRecall,
    MulticlassF1Score,
    MulticlassConfusionMatrix,
)

torch.manual_seed(42)

DISPOSITIVO = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
```

```
)
print("Dispositivo:", DISPOSITIVO)
```

```
Dispositivo: mps
```

---

1. Datos: CIFAR-10 a 224x224

ResNet y EfficientNet, tal como se definen en `torchvision.models`, esperan entradas de 224x224 pixeles (el tamano estandar de ImageNet en el que se disenaron y evaluaron originalmente). Redimensionamos CIFAR-10 (32x32 nativo) a ese tamano.

In [3]:
```
MEDIA_IMAGENET = (0.485, 0.456, 0.406)
STD_IMAGENET = (0.229, 0.224, 0.225)

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(MEDIA_IMAGENET, STD_IMAGENET),
])

# CARGAR IMAGENES DE ENTRENAMIENTO Y PRUEBA Y GENERAR UN SUBCONJUNTO
trainset_completo = torchvision.datasets.CIFAR10(root='./data',
train=True, download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform)

# Dividir el conjunto de entrenamiento en entrenamiento y validación
TAM_TRAIN = 45000
TAM_VAL = 5000

train_subset, val_subset = random_split(trainset_completo, [TAM_TRAIN,
TAM_VAL])

nombres_clases = trainset_completo.classes
print("Clases:", nombres_clases)
print("Tamano del set de entrenamiento completo:",
len(trainset_completo))
print("Tamano del set de prueba (testset):", len(testset))
```

```
Clases: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog',
'horse', 'ship', 'truck']
Tamano del set de entrenamiento completo: 50000
Tamano del set de prueba (testset): 10000
```

1.1 Subconjuntos de trabajo

In [4]:
```
BATCH_SIZE = 32
train_loader = DataLoader(train_subset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2)
val_loader = DataLoader(val_subset, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2)
test_loader = DataLoader(testset, batch_size=64, shuffle=False,
num_workers=2)

print(f"Entrenamiento: {len(train_subset)} imagenes  |  Validacion:
{len(val_subset)}  |  Prueba: {len(testset)}")
```

```
def desnormalizar(img_tensor):
    """Invierte la normalizacion para poder visualizar la imagen."""
    media = torch.tensor(MEDIA_IMAGENET).view(3, 1, 1)
    std = torch.tensor(STD_IMAGENET).view(3, 1, 1)
    return (img_tensor * std + media).clamp(0, 1)


imagenes, etiquetas = next(iter(train_loader))
fig, axs = plt.subplots(1, 6, figsize=(12, 2.5))
for i, ax in enumerate(axs):
    ax.imshow(desnormalizar(imagenes[i]).permute(1, 2, 0))
    ax.set_title(nombres_clases[etiquetas[i]], fontsize=10)
    ax.axis("off")
plt.suptitle("Ejemplos del subconjunto de entrenamiento (redimensionados
a 224x224)")
plt.show()
```

```
Entrenamiento: 45000 imagenes  |  Validacion: 5000  |  Prueba: 10000
```



---

2. Construir ResNet-18 y EfficientNet-B0

Tomamos las arquitecturas de `torchvision.models` con pesos preentrenados en ImageNet (`weights="DEFAULT"`) y adaptamos unicamente la ultima capa para que clasifique entre las 10 clases de CIFAR-10 en vez de las 1000 de ImageNet — el resto de la arquitectura, y sus pesos preentrenados, se conservan exactamente como en 1.3.

Ver : https://docs.pytorch.org/vision/main/models.html

Ejemplo:
https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html

In [5]:
```
# COMPLETAR construir_resnet18(n_clases=10)

def construir_resnet18(n_clases=10):
    modelo = models.resnet18(weights="DEFAULT")
    modelo.fc = nn.Linear(modelo.fc.in_features, n_clases)
    return modelo


# COMPLETAR construir_efficientnet_b0(n_clases=10)
# Unico cambio estructural permitido: reemplazar la ultima capa
(classifier[1])
def construir_efficientnet_b0(n_clases=10):
    modelo = models.efficientnet_b0(weights="DEFAULT")
```

```
    modelo.classifier[1] = nn.Linear(modelo.classifier[1].in_features,
n_clases)
    return modelo


def contar_parametros(modelo):
    """Numero total de parametros entrenables (para el reporte, punto
5)."""
    return sum(p.numel() for p in modelo.parameters() if p.requires_grad)


# Conteo de parametros de cada arquitectura (Parte 2 / reporte punto 5)
modelo_resnet = construir_resnet18(n_clases=10)
modelo_efficientnet = construir_efficientnet_b0(n_clases=10)
print(f"ResNet-18:       {contar_parametros(modelo_resnet):,}
parametros")
print(f"EfficientNet-B0: {contar_parametros(modelo_efficientnet):,}
parametros")
```

```
ResNet-18:       11,181,642 parametros
EfficientNet-B0: 4,020,358 parametros
```

---

3. Metricas con TorchMetrics

Mismo patron que en la Practica 4 original: `update()` por lote, `compute()` al final de la epoca, `reset()` antes de la siguiente. Usamos `average="macro"` (promedio simple entre las 10 clases)

ver: https://meta-pytorch.org/torcheval/main/torcheval.metrics.html

Ejemplo :

"accuracy": MulticlassAccuracy(num_classes=n_clases, average="macro").to(DISPOSITIVO),

In [6]:
```
def crear_metricas(n_clases=10):
    return {
         #Completar
        "accuracy": MulticlassAccuracy(num_classes=n_clases,
average="macro").to(DISPOSITIVO),
        "precision": MulticlassPrecision(num_classes=n_clases,
average="macro").to(DISPOSITIVO),
        "recall": MulticlassRecall(num_classes=n_clases,
average="macro").to(DISPOSITIVO),
        "f1": MulticlassF1Score(num_classes=n_clases,
average="macro").to(DISPOSITIVO),
    }


def correr_epoca(modelo, loader, perdida_fn, metricas, optimizador=None):
    """
    Corre una epoca completa. Si optimizador no es None, entrena
(backward + step);
    si es None, solo evalua (torch.no_grad()).
    """
    modo_entrenamiento = optimizador is not None
    modelo.train() if modo_entrenamiento else modelo.eval()
```

```
    for m in metricas.values():
        m.reset()

    perdida_acumulada, n_lotes = 0.0, 0
    contexto = torch.enable_grad() if modo_entrenamiento else
torch.no_grad()

    with contexto:
        for imagenes, etiquetas in loader:
            imagenes, etiquetas = imagenes.to(DISPOSITIVO),
etiquetas.to(DISPOSITIVO)

            if modo_entrenamiento:
                optimizador.zero_grad()

            salida = modelo(imagenes)
            perdida = perdida_fn(salida, etiquetas)

            if modo_entrenamiento:
                perdida.backward()
                optimizador.step()

            perdida_acumulada += perdida.item()
            n_lotes += 1

            preds = salida.argmax(dim=1)
            for m in metricas.values():
                m.update(preds, etiquetas)

    resultados = {nombre: m.compute().item() for nombre, m in
metricas.items()}
    resultados["loss"] = perdida_acumulada / n_lotes
    return resultados
```

---

4. Entrenar ambos modelos en igualdad de condiciones

Para que la comparacion sea tecnicamente valida, ambos modelos usan **los mismos datos, el mismo numero de epocas, el mismo optimizador y el mismo learning rate**. Documentamos estos hiperparametros aqui porque los van a necesitar tal cual para el reporte de diseno (Parte 7).

In [7]:
```
# ===== HIPERPARAMETROS (identicos para ambos modelos — seccion 3 del
PDF) =====
LR = 1e-4                  # learning rate
NOMBRE_OPTIMIZADOR = "Adam"
N_EPOCAS = 5
# BATCH_SIZE = 32 y TAM_TRAIN = 45000 ya estan definidos en la seccion 1

print(f"Learning rate : {LR}")
print(f"Optimizador   : {NOMBRE_OPTIMIZADOR}")
print(f"Epocas        : {N_EPOCAS}")
print(f"Batch size    : {BATCH_SIZE}")
print(f"Tam. train    : {TAM_TRAIN}  |  Tam. val: {TAM_VAL}  |  Tam.
```

```
test: {len(testset)}")
print("Pesos         : preentrenados en ImageNet (weights=\"DEFAULT\")")
print(f"Dispositivo   : {DISPOSITIVO}")


def entrenar_modelo(modelo, nombre_modelo):
    """
    Entrena `modelo` con las condiciones fijas de arriba y devuelve
    (modelo_entrenado, historial_como_DataFrame).
    Cada fila del historial trae: epoca, duracion_s, train_*/val_* (loss
y metricas).
    """
    modelo = modelo.to(DISPOSITIVO)
    perdida_fn = nn.CrossEntropyLoss()
    optimizador = optim.Adam(modelo.parameters(), lr=LR)
    metricas = crear_metricas(n_clases=10)

    historial = []
    print(f"\n=== {nombre_modelo} | parametros:
{contar_parametros(modelo):,} ===")

    for epoca in range(1, N_EPOCAS + 1):
        t0 = time.time()
        res_train = correr_epoca(modelo, train_loader, perdida_fn,
metricas, optimizador)
        res_val = correr_epoca(modelo, val_loader, perdida_fn, metricas,
optimizador=None)
        duracion = time.time() - t0

        fila = {"epoca": epoca, "duracion_s": round(duracion, 1)}
        fila.update({f"train_{k}": v for k, v in res_train.items()})
        fila.update({f"val_{k}": v for k, v in res_val.items()})
        historial.append(fila)

        print(
            f"Epoca {epoca}/{N_EPOCAS} | {duracion:6.1f}s | "
            f"train loss {res_train['loss']:.4f} acc
{res_train['accuracy']:.3f} | "
            f"val loss {res_val['loss']:.4f} acc
{res_val['accuracy']:.3f}"
        )

    return modelo, pd.DataFrame(historial)


# ===== Entrenamiento de ambos modelos bajo las mismas condiciones =====
# (modelo_resnet y modelo_efficientnet se construyeron en la seccion 2)
modelo_resnet, hist_resnet = entrenar_modelo(modelo_resnet, "ResNet-18")
modelo_efficientnet, hist_efficientnet =
entrenar_modelo(modelo_efficientnet, "EfficientNet-B0")

df_historial = pd.concat(
    [hist_resnet.assign(modelo="ResNet-18"),
hist_efficientnet.assign(modelo="EfficientNet-B0")],
    ignore_index=True,
)
df_historial.round(4)
```

```
Learning rate : 0.0001
Optimizador   : Adam
Epocas        : 5
Batch size    : 32
Tam. train    : 45000  |  Tam. val: 5000  |  Tam. test: 10000
Pesos         : preentrenados en ImageNet (weights="DEFAULT")
Dispositivo   : mps

=== ResNet-18 | parametros: 11,181,642 ===
Epoca 1/5 |  155.1s | train loss 0.3466 acc 0.886 | val loss 0.2112 acc
0.929
Epoca 2/5 |  149.1s | train loss 0.1281 acc 0.958 | val loss 0.1819 acc
0.938
Epoca 3/5 |  148.1s | train loss 0.0760 acc 0.976 | val loss 0.1976 acc
0.934
Epoca 4/5 |  148.2s | train loss 0.0514 acc 0.983 | val loss 0.1769 acc
0.945
Epoca 5/5 |  149.2s | train loss 0.0483 acc 0.984 | val loss 0.1861 acc
0.941

=== EfficientNet-B0 | parametros: 4,020,358 ===
Epoca 1/5 |  300.4s | train loss 0.4238 acc 0.868 | val loss 0.1385 acc
0.952
Epoca 2/5 |  291.3s | train loss 0.1493 acc 0.950 | val loss 0.1122 acc
0.959
Epoca 3/5 |  289.8s | train loss 0.0915 acc 0.969 | val loss 0.1190 acc
0.961
Epoca 4/5 |  289.7s | train loss 0.0643 acc 0.979 | val loss 0.1343 acc
0.954
Epoca 5/5 |  366.2s | train loss 0.0516 acc 0.983 | val loss 0.1173 acc
0.964
```

Out[7]:

	epoca	duracion_s	train_accuracy	train_precision	train_recall	train_f1	train_lo
0	1	155.1	0.8858	0.8856	0.8858	0.8856	0.34
1	2	149.1	0.9580	0.9580	0.9580	0.9580	0.12
2	3	148.1	0.9759	0.9759	0.9759	0.9759	0.07
3	4	148.2	0.9835	0.9835	0.9835	0.9835	0.05
4	5	149.2	0.9838	0.9838	0.9838	0.9838	0.04
5	1	300.4	0.8680	0.8678	0.8680	0.8678	0.42
6	2	291.3	0.9505	0.9504	0.9505	0.9505	0.14
7	3	289.8	0.9691	0.9691	0.9691	0.9691	0.09
8	4	289.7	0.9786	0.9786	0.9786	0.9786	0.06
9	5	366.2	0.9832	0.9832	0.9832	0.9832	0.05

5. Curvas de aprendizaje: ResNet-18 vs. EfficientNet-B0

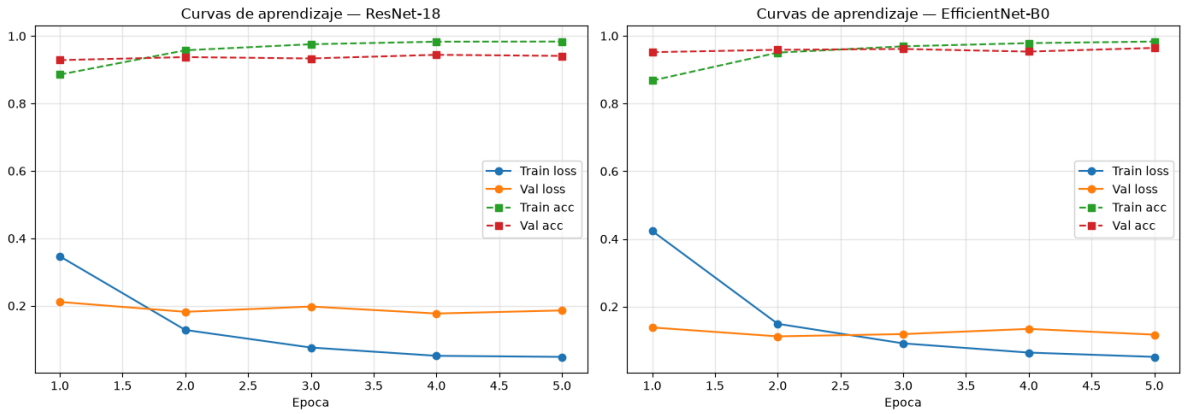
Mismo diagnostico que en 1.4: comparen el gap train/val de cada modelo, y ahora tambien comparenlos *entre si* — ¿cual generaliza mejor con el mismo presupuesto de datos y epocas?

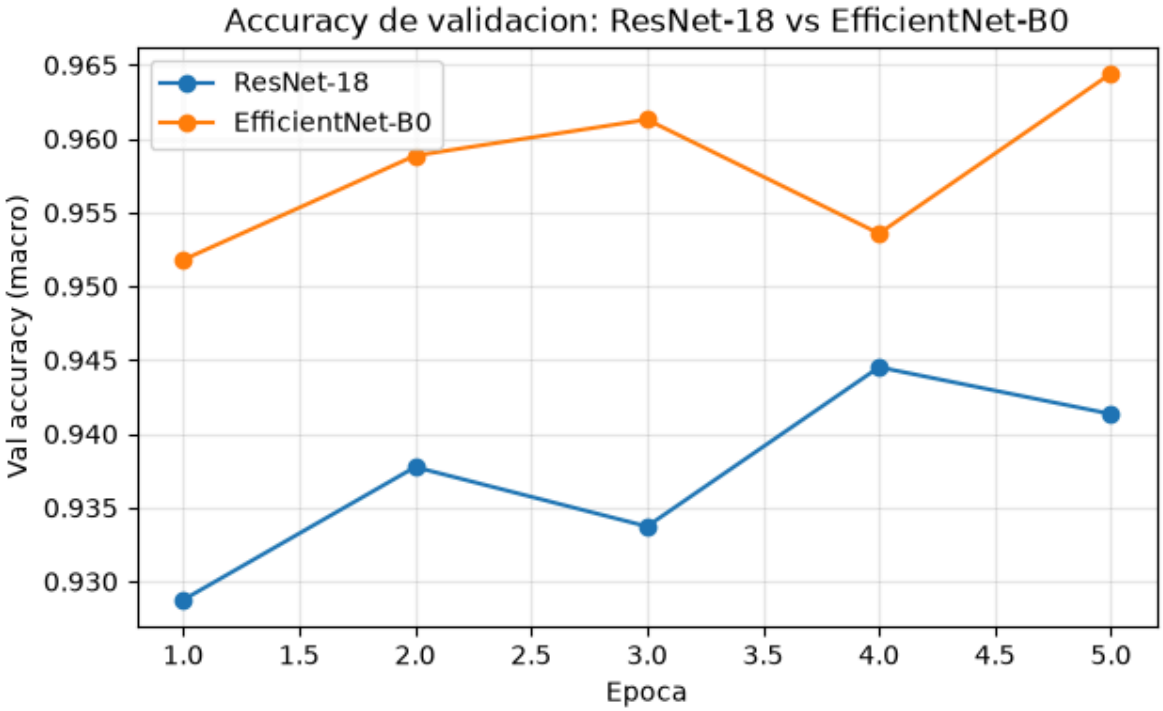
In [8]:
```
# ===== Curvas de aprendizaje: loss y accuracy (train y validacion) =====
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

for ax, hist, nombre in [
    (axs[0], hist_resnet, "ResNet-18"),
    (axs[1], hist_efficientnet, "EfficientNet-B0"),
]:
    ax.plot(hist["epoca"], hist["train_loss"], marker="o", label="Train
loss")
    ax.plot(hist["epoca"], hist["val_loss"], marker="o", label="Val
loss")
    ax.plot(hist["epoca"], hist["train_accuracy"], marker="s",
linestyle="--", label="Train acc")
    ax.plot(hist["epoca"], hist["val_accuracy"], marker="s", linestyle="-
-", label="Val acc")
    ax.set_xlabel("Epoca")
    ax.set_title(f"Curvas de aprendizaje — {nombre}")
    ax.legend()
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()


# Comparacion directa del accuracy de validacion entre ambos modelos
plt.figure(figsize=(7, 4))
plt.plot(hist_resnet["epoca"], hist_resnet["val_accuracy"], marker="o",
label="ResNet-18")
plt.plot(hist_efficientnet["epoca"], hist_efficientnet["val_accuracy"],
marker="o", label="EfficientNet-B0")
plt.xlabel("Epoca"); plt.ylabel("Val accuracy (macro)")
plt.title("Accuracy de validacion: ResNet-18 vs EfficientNet-B0")
plt.legend(); plt.grid(alpha=0.3)
plt.show()
```





---

6. Reporte final sobre el set de prueba

Evaluamos ambos modelos sobre las 10,000 imagenes de prueba de CIFAR-10

In [9]:
```
def evaluar_en_prueba(modelo, nombre_modelo):
    metrica_acc = MulticlassAccuracy(num_classes=10,
average="macro").to(DISPOSITIVO)
    metrica_prec = MulticlassPrecision(num_classes=10,
average="macro").to(DISPOSITIVO)
    metrica_rec = MulticlassRecall(num_classes=10,
average="macro").to(DISPOSITIVO)
    metrica_f1 = MulticlassF1Score(num_classes=10,
average="macro").to(DISPOSITIVO)
    metrica_matriz =
MulticlassConfusionMatrix(num_classes=10).to(DISPOSITIVO)

    modelo.eval()
    with torch.no_grad():
        for imagenes, etiquetas in test_loader:
            imagenes, etiquetas = imagenes.to(DISPOSITIVO),
etiquetas.to(DISPOSITIVO)
            preds = modelo(imagenes).argmax(dim=1)
            for m in [metrica_acc, metrica_prec, metrica_rec, metrica_f1,
metrica_matriz]:
                m.update(preds, etiquetas)

    resumen = {
        "modelo": nombre_modelo,
        "accuracy": metrica_acc.compute().item(),
        "precision": metrica_prec.compute().item(),
        "recall": metrica_rec.compute().item(),
        "f1": metrica_f1.compute().item(),
    }
    return resumen, metrica_matriz.compute().cpu().numpy()
```

```
resumen_resnet, matriz_resnet = evaluar_en_prueba(modelo_resnet, "ResNet-18")
resumen_efficientnet, matriz_efficientnet = evaluar_en_prueba(modelo_efficientnet, "EfficientNet-B0")

df_comparativa = pd.DataFrame([resumen_resnet, resumen_efficientnet]).round(3)
df_comparativa
```

Out[9]:

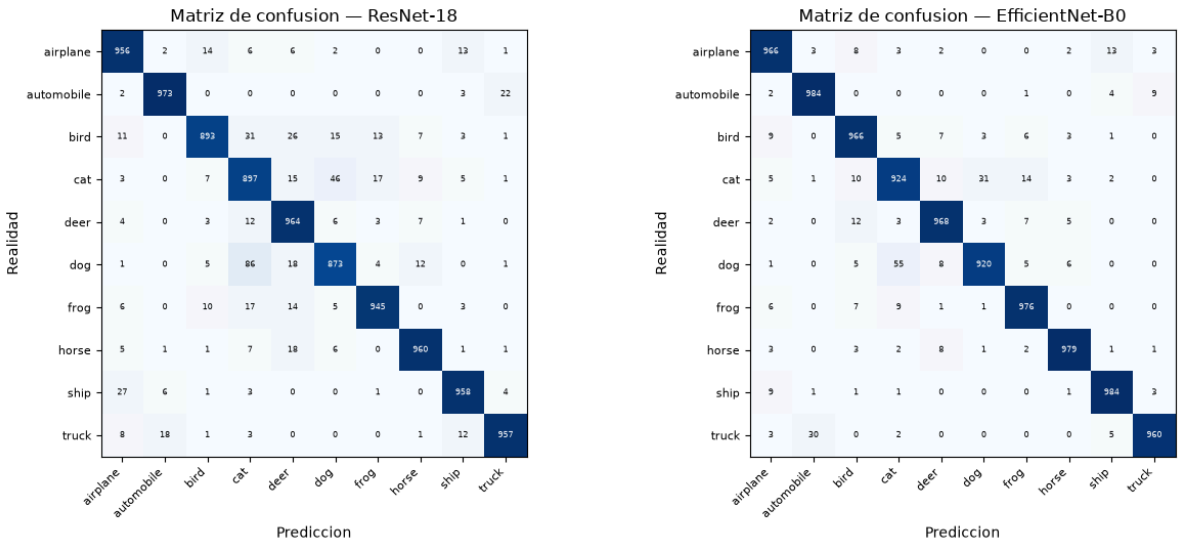
	modelo	accuracy	precision	recall	f1
0	ResNet-18	0.938	0.938	0.938	0.938
1	EfficientNet-B0	0.963	0.963	0.963	0.963

In [10]:

```
fig, axs = plt.subplots(1, 2, figsize=(13, 5.5))

for ax, matriz, nombre in [(axs[0], matriz_resnet, "ResNet-18"), (axs[1], matriz_efficientnet, "EfficientNet-B0")]:
    im = ax.imshow(matriz, cmap="Blues")
    ax.set_xticks(range(10)); ax.set_xticklabels(nombres_clases, rotation=45, ha="right", fontsize=8)
    ax.set_yticks(range(10)); ax.set_yticklabels(nombres_clases, fontsize=8)
    ax.set_xlabel("Prediccion"); ax.set_ylabel("Realidad")
    ax.set_title(f"Matriz de confusion — {nombre}")
    for i in range(10):
        for j in range(10):
            color = "white" if matriz[i, j] > matriz.max() / 2 else "black"
            ax.text(j, i, str(int(matriz[i, j])), ha="center", va="center", color=color, fontsize=6)

plt.tight_layout()
plt.show()
```



---

7. Reporte de diseño

Evidencia **"Reporte de diseño"** de la Unidad 1. Todas las cifras provienen de las celdas ejecutadas arriba (semilla fija `torch.manual_seed(42)`; los valores pueden variar ligeramente entre ejecuciones).

1. Arquitectura y justificación

> Recomendamos **EfficientNet-B0**. En el conjunto de prueba (10,000 imágenes) obtuvo accuracy / precision / recall / F1 = **0.963** frente a **0.938** de ResNet-18 (+2.5 puntos porcentuales). También generalizó mejor en validación (mejor val accuracy 0.964 vs 0.945; val loss final 0.117 vs 0.186) con un gap train/val menor (ver punto 4), y usa menos de la mitad de los parámetros. Su desventaja es el costo de cómputo por época (ver punto 3). Como el accuracy macro coincide con el recall macro y CIFAR-10 está balanceado (1,000 imágenes por clase en prueba), las cuatro métricas son casi idénticas.

2. Hiperparámetros usados

> Learning rate: `1e−4` · Optimizador: `Adam` · Épocas: `5` · Batch size: `32` · Tamaño del subconjunto de entrenamiento: `45000` (validación: `5000`, prueba: `10000`) · Pesos iniciales: preentrenados (ImageNet, `weights="DEFAULT"`) con normalización de ImageNet y `Resize` a 224×224 · Pérdida: `CrossEntropyLoss` · Métricas: torchmetrics, promedio macro. Idénticos para ambos modelos.

3. Estrategia de entrenamiento

> Hardware: **GPU Apple Silicon (backend MPS de PyTorch)**, ejecución local con entorno `uv` · Duración por época: ResNet-18 ≈ **150 s/época** (148–155 s); EfficientNet-B0 ≈ **290 s/época** (290–300 s en las épocas 1–4 y 366 s en la 5, probablemente por carga del sistema). EfficientNet-B0 tarda ~2× más por época a pesar de tener menos parámetros, porque sus convoluciones depthwise y bloques de expansión/squeeze-excitation son más costosos en tiempo real que en FLOPs.
>
> Si tuviéramos más tiempo, cambiaríamos primero: **aumento de datos (flips/crops aleatorios) y un scheduler de learning rate** (p. ej. coseno), porque ResNet-18 muestra señales de sobreajuste y la val loss de ambos oscila entre épocas; después más épocas con *early stopping* sobre la val loss.

4. Diagnóstico de las curvas

> - **ResNet-18:** train loss baja de 0.347 a 0.048 y train acc sube a 0.984, pero la val loss se estanca en ~0.18–0.20 (mínimo 0.177 en la época 4, 0.186 al final) y la val acc en ~0.94. Gap train/val acc final ≈ **4.3 pp** → **sobreajuste leve a moderado**: aprende el entrenamiento pero deja de mejorar en validación desde la época 2.

- **EfficientNet-B0:** train loss 0.424 → 0.052, train acc 0.983; val loss 0.139 → 0.117 (mínimo 0.112 en la época 2) y val acc 0.952 → 0.964. Gap final ≈ **1.9 pp** → **ajuste saludable**, con ligero inicio de sobreajuste (la val loss sube a 0.134 en la época 4 y vuelve a bajar). No hay subajuste en ninguno: ambos superan 0.98 de acc en entrenamiento.
- EfficientNet-B0 generaliza mejor con el mismo presupuesto de datos y épocas.

5. Capacidad vs. eficiencia

Parámetros: ResNet-18 = `11,181,642` · EfficientNet-B0 = `4,020,358` (≈ 2.8× menos).

EfficientNet-B0 logra **+2.5 pp de accuracy en prueba con 64% menos parámetros**, por lo que ofrece la mejor relación desempeño/parámetros (y menor tamaño en memoria/disco). Sin embargo, menos parámetros no implica menos tiempo: en nuestro hardware cuesta ~2× el tiempo de entrenamiento por época. Si el recurso crítico es el tiempo de cómputo, ResNet-18 es más barato (con ~94% de accuracy); si lo es la precisión o el tamaño del modelo, EfficientNet-B0 es superior. ResNet-18 tiene mayor capacidad bruta pero la usa para memorizar (sobreajuste), no para generalizar mejor.

6. Errores más frecuentes

- **ResNet-18:** el par más confundido es **dog ↔ cat**: 86 perros predichos como gato y 46 gatos como perro (132 errores en total). Siguen automobile ↔ truck (22 + 18) y ship → airplane (27).
- **EfficientNet-B0:** también **dog ↔ cat**: 55 perros como gato y 31 gatos como perro (86 en total), un 35% menos que ResNet-18. El segundo par es truck → automobile (30) y automobile → truck (9).
- **Explicación:** gatos y perros son animales cuadrúpedos con pelaje, tamaño y poses similares, con fondos domésticos parecidos y gran variabilidad intra-clase; además las imágenes originales son de 32×32 y al ampliarlas a 224×224 se pierde el detalle fino (forma de orejas, hocico) que los distingue. Automóvil y camión comparten forma general (ruedas, carrocería, vistas laterales) y difieren sobre todo en escala, que es difícil de apreciar en imágenes recortadas y de baja resolución.

---

8. Cierre y entregables del mini proyecto

Checklist de entrega

- [x] Notebook ejecutado de principio a fin, sin errores, con las celdas de ambos modelos corridas.
- [x] Tabla comparativa final (Parte 6) con accuracy, precision, recall y F1 de ambos modelos.
- [x] Graficas de curvas de entrenamiento (Parte 5) y matrices de confusion (Parte 6).
- [x] Reporte de diseno (Parte 7) completado por el equipo.
- [x] Hiperparametros documentados tal como quedaron configurados al momento de generar los resultados finales.

Recursos citados

- Katanforoosh, K., & Ng, A. *Stanford CS230* — Section 8, "Advanced Evaluation Metrics". cs230.stanford.edu/section/8
- TorchMetrics documentation — *Classification: Accuracy*. torchmetrics.readthedocs.io/en/v0.9.1/classification/accuracy.html
- Ultralytics. *Consejos y buenas practicas para entrenar modelos*. docs.ultralytics.com/es/guides/model-training-tips
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. arXiv:1512.03385.
- Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. ICML 2019.
- Ayyadevara, K., & Reddy, Y. (2024). *Modern Computer Vision with PyTorch*. Packt Publishing.